GitHub Pull Requests

Complete Course Notes

Pull Request Lifecycle, Reviews, Roles & Permissions

1. What Is a Pull Request?

A Pull Request (PR) is a formal mechanism in GitHub for proposing changes from one branch into another. It is the core collaboration tool that enables teams to review, discuss, and merge code safely.

Key characteristics of a Pull Request:

- A PR is NOT a Git feature — it is a GitHub feature built on top of Git
- It represents a request to pull changes from one branch into another (typically feature branch → main/develop)
- It creates a dedicated space for discussion, inline code comments, and review
- It provides a full diff view showing exactly what changed between branches
- It can trigger automated checks (CI/CD, tests, linting) before merging
- It creates a permanent record of why and how changes were made

Why Pull Requests matter:

- Enforce code quality — no direct pushes to protected branches
- Enable knowledge sharing — reviewers learn the codebase through review
- Catch bugs early — multiple eyes spot issues before they reach production
- Document decisions — discussions and comments explain the reasoning behind changes
- Support compliance — audit trail of all changes and who approved them

2. The 6-Step Pull Request Lifecycle

Every Pull Request follows a consistent lifecycle regardless of team size or project type:

Step	Action	Who
1	Checkout — create a branch	Developer
2	Commit — make and push changes	Developer
3	Create — open the Pull Request	Developer
4	Review — examine the changes	Team / Reviewers
5	Approve — formally sign off	Reviewer / Maintainer
6	Merge — integrate into target branch	Developer / Maintainer

Step 1: Checkout — Create a Branch

Before making any changes, create a dedicated feature branch. This keeps your work isolated from the main codebase.

```
git checkout -b feature/my-new-feature
```

Or using the newer syntax:

```
git switch -c feature/my-new-feature
```

Branch naming conventions:

- feature/description — for new features
- bugfix/issue-number — for bug fixes
- hotfix/critical-fix — for urgent production fixes
- chore/task-name — for maintenance tasks (dependency updates, refactoring)
- docs/update-readme — for documentation-only changes

Step 2: Commit — Make and Push Changes

Make your code changes, stage them, commit with a meaningful message, and push to the remote repository.

```
git add .
git commit -m "feat: add user authentication with JWT"
git push origin feature/my-new-feature
```

Best practices for commits in a PR workflow:

- Write descriptive commit messages (use conventional commits: feat:, fix:, docs:, chore:)
- Make small, focused commits — each commit should represent one logical change
- Avoid mixing unrelated changes in a single commit
- Use git push --set-upstream origin branch-name on first push

Step 3: Create — Open the Pull Request

After pushing your branch, navigate to GitHub to open the PR. GitHub often shows a banner suggesting you create a PR from your recently pushed branch.

When creating a PR, fill in:

- Title — clear, concise summary of the change (e.g., 'Add JWT-based user authentication')
- Description — explain the why, what changed, how to test, and any relevant context
- Reviewers — tag specific team members whose review you need
- Assignees — who is responsible for this PR (usually yourself)
- Labels — categorize the PR (bug, enhancement, documentation, etc.)

- Milestone — link to a project milestone if applicable
- Linked Issues — use 'Closes #123' or 'Fixes #456' to auto-close issues on merge

Draft Pull Requests:

- Marked as Draft to signal the PR is a work in progress
- Reviewers can see it but know it is not ready for formal review
- Useful for early feedback, visibility, or CI/CD checks while still developing
- Convert to Ready for Review when complete

Step 4: Review — Examine the Changes

Reviewers examine the PR in detail. GitHub provides several tools for thorough review:

The Files Changed Tab

Shows a diff (difference) view of every file modified:

- Green lines (+ prefix) — lines that were added
- Red lines (- prefix) — lines that were removed
- Unchanged lines shown for context

Inline Comments

Reviewers can leave comments on specific lines of code:

- Click the + icon that appears when hovering over a line
- Type your comment — ask questions, suggest improvements, flag issues
- Start a review to batch multiple comments before submitting
- Add a suggestion — propose a specific code change inline using the suggestion block:

```
```suggestion
const newCode = correctImplementation();
```
```

The author can accept suggestions with one click, which commits the change automatically.

Review Types (When Submitting a Review)

Review Type	When to Use	Effect on PR
Comment	General feedback, questions, discussion	No blocking effect
Approve	Code is ready to merge as-is	Satisfies approval requirement
Request	Issues must be fixed before merging	Blocks merging until resolved

Review Type	When to Use	Effect on PR
Changes		

Review Checklist — What Good Reviewers Check

- Correctness — does the code do what the PR description says?
- Logic — are there edge cases or bugs?
- Style — does it follow team coding standards and conventions?
- Tests — are there unit/integration tests? Do existing tests still pass?
- Security — any vulnerabilities, exposed secrets, or unsafe inputs?
- Performance — any obvious inefficiencies or unnecessary operations?
- Documentation — are public APIs, functions, or complex logic documented?
- Scope — does the PR stay focused, or does it mix unrelated changes?

Step 5: Approve — Formally Sign Off

Once a reviewer is satisfied with the changes, they submit a formal Approval. Depending on repository settings, a minimum number of approvals may be required before merging.

Branch protection rules related to approvals:

- Require X number of approving reviews before merging (e.g., 1 or 2)
- Dismiss stale approvals — if new commits are pushed after approval, approvals are reset
- Require review from code owners — designated experts must approve changes in their area
- Restrict who can dismiss pull request reviews

Step 6: Merge — Integrate into the Target Branch

After receiving required approvals (and passing all checks), the PR can be merged. GitHub offers three merge strategies:

Merge Strategy	Description	Commit History	Best For
Merge Commit	Creates a merge commit combining both branches	Preserves full history of all commits	Teams who want full audit trail
Squash and Merge	Combines all PR commits into one commit on target branch	Clean single-commit history	Feature branches with many WIP commits

Merge Strategy	Description	Commit History	Best For
Rebase and Merge	Replays PR commits on top of target branch (no merge commit)	Linear history, no merge commits	Teams who prefer linear history

After merging:

- Optionally delete the feature branch (GitHub offers a Delete Branch button)
- Any linked issues are automatically closed if 'Closes #issue' was in the PR description
- The merge appears in the repository's commit history and Insights

3. Repository Roles and Permissions

GitHub provides a granular permission system to control who can do what in a repository. Understanding roles is essential for managing Pull Requests effectively.

3.1 Repository Permission Levels

Role	Description	Key Capabilities
Read	View-only access	View code, clone repo, read issues and PRs, leave comments
Triage	Manage issues without write access	All Read permissions + manage issues and PRs (label, close, reopen) — cannot push code
Write	Contribute to the repository	All Triage permissions + push to non-protected branches, create/merge PRs, manage releases
Maintain	Manage repo without destructive actions	All Write permissions + manage branch protection, merge to protected branches, manage repository settings (limited)
Admin	Full control	All Maintain permissions + manage collaborators, delete repo, change visibility, bypass all protections

3.2 Key Admin Capabilities

Admins (and Maintainers) control the behavior of Pull Requests through repository settings:

Branch Protection Rules

- Require pull request reviews before merging — enforce the PR workflow on protected branches
- Require status checks to pass — CI/CD must succeed before merge is allowed

- Require conversation resolution — all PR comments must be marked as resolved
- Require signed commits — only cryptographically signed commits accepted
- Require linear history — prevent merge commits (enforce squash or rebase)
- Restrict who can push to the branch — limit direct pushes to specific teams or users
- Allow force pushes — optionally permit force push for specific users or everyone
- Allow deletions — control who can delete the protected branch

CODEOWNERS File

Admins can define a CODEOWNERS file at the root, .github/, or docs/ directory. It automatically assigns reviewers based on which files were changed.

```
# CODEOWNERS syntax
# File pattern    @owner
*.js              @frontend-team
*.py              @backend-team
/docs/            @documentation-team
src/auth/         @security-team @lead-developer
```

When a PR modifies files matching a pattern, the listed owners are automatically requested as reviewers. If branch protection requires CODEOWNERS review, these owners must approve before merging.

3.3 PR Templates

Admins can create Pull Request templates to ensure contributors provide consistent information:

```
Location: .github/pull_request_template.md
```

A typical PR template includes:

- ## Description — what does this PR do and why?
- ## Type of Change — bug fix, new feature, breaking change, documentation
- ## How Has This Been Tested? — describe test scenarios
- ## Checklist — [ ] tests added, [ ] docs updated, [ ] reviewed for security
- ## Related Issues — Closes #issue-number

For multiple templates, create a .github/PULL_REQUEST_TEMPLATE/ folder with multiple .md files. GitHub presents a choice when creating a PR.

3.4 Merge Controls

Admins configure which merge strategies are available in the repository settings:

- Allow merge commits — enable/disable standard merge commits
- Allow squash merging — enable/disable squash and merge
- Allow rebase merging — enable/disable rebase and merge
- Auto-delete head branches — automatically delete feature branches after merge
- Auto-merge — allow PRs to automatically merge once all conditions are met

4. Working With Pull Requests — Practical Workflows

4.1 Keeping a PR Up to Date

When the target branch receives new commits while your PR is open, your branch may fall behind. You have two options:

Option A: Merge the target branch into your feature branch

```
git checkout feature/my-branch
git merge main
git push origin feature/my-branch
```

Option B: Rebase your feature branch onto the target branch

```
git checkout feature/my-branch
git rebase main
git push --force-with-lease origin feature/my-branch
```

(Note: force push after rebase — coordinate with team to avoid overwriting others' work)

GitHub also provides an 'Update branch' button in the PR interface that performs a merge automatically.

4.2 Resolving Review Feedback

When a reviewer requests changes:

1. Read all comments carefully before making changes
2. Make the requested changes in your local branch
3. Commit and push the changes
4. Reply to each comment explaining what you did (or why you disagree)
5. Re-request review from the reviewer using the re-request button

4.3 Handling Merge Conflicts

Merge conflicts occur when the same lines were modified in both branches. GitHub shows a conflict warning in the PR.

Resolving via GitHub Web UI:

6. Click 'Resolve conflicts' in the PR interface
7. GitHub shows the conflicting file with conflict markers
8. Edit the file to keep the correct content, remove conflict markers
9. Click 'Mark as resolved' then 'Commit merge'

Resolving locally:

```
git checkout feature/my-branch
git merge main
# Edit conflicting files, remove <<<<<<, =======, >>>>>>> markers
git add .
git commit -m "resolve merge conflicts"
git push origin feature/my-branch
```

4.4 PR Best Practices

- Keep PRs small and focused — easier to review, faster to merge
- Write a clear description — explain the why, not just the what
- Link related issues — creates traceability between work items and code
- Self-review before requesting review — catch obvious issues yourself first
- Respond to all comments — even if just acknowledging or marking as resolved
- Don't merge your own PR (when possible) — have someone else approve and merge
- Use draft PRs for early visibility — keeps the team informed without demanding review
- Clean up branches after merge — delete feature branches to keep the repo tidy

5. Quick Reference Summary

Concept	Key Point
Pull Request	GitHub feature (not Git) for proposing, reviewing, and merging code changes

Concept	Key Point
Step 1: Checkout	Create a feature branch: git checkout -b feature/name
Step 2: Commit	Make changes, git add, git commit, git push
Step 3: Create	Open PR on GitHub with title, description, reviewers, linked issues
Step 4: Review	Reviewers examine diff, leave inline comments, suggest changes
Review Types	Comment (neutral), Approve (green light), Request Changes (blocking)
Step 5: Approve	Formal sign-off; branch protection may require minimum number of approvals
Step 6: Merge	Merge commit (full history), Squash (clean), or Rebase (linear)
Roles	Read → Triage → Write → Maintain → Admin (increasing permissions)
Branch Protection	Admin rules: require reviews, status checks, signed commits, etc.
CODEOWNERS	.github/CODEOWNERS auto-assigns reviewers based on changed files
PR Templates	.github/pull_request_template.md standardizes PR descriptions
Draft PRs	Signal WIP; get early feedback without formal review request
Merge Conflicts	Resolve via GitHub UI or locally with git merge + editing conflict markers